# UNIVERSITY OF ENGINEERING AND TECHNOLOGY MARDAN

| | |
|---|---|
| **Name** | **ASIM AYAZ** |
| **Registration No#** | **25MDBCS804** |
| **Assignment** | **Lab Assignment 10** |
| **Submitted to** | **Mam Alishba** |
| **Semester** | **2nd** |
| **Date** | **17-04-2026** |

# Task 1:

# Implement the concept of abstract class in C++.

# Program:

C:\Users\MCS\OneDrive\Desktop\oop lab 10 1.cpp - Dev-C++ 5.11

File Edit Search View Project Execute Tools AStyle Window Help

TDM-GCC 4.9.2 64-bit Release

(globals)

Project [*] oop lab 10 1.cpp

```cpp
#include<iostream>
using namespace std;

class animal{
    public:
        virtual void sound()=0; // this is pure virtual function , and any class with atleast //
        //1 vital function becomes an abstract class//

};
class dog:public animal{
    public:

        void sound(){
            cout<<"........DOG = BARKS........"<<endl;
        }
};
int main()
{
    dog d1;
    d1.sound();
    return 0;
}
```

# Output:

C:\Users\MCS\OneDrive\Desktop\oop lab 10 1.exe

```
........DOG = BARKS........

--------------------------------
Process exited after 0.02838 seconds with return value 0
Press any key to continue . . .
```

# TASK 2:

Consider an abstract class Duck, representing all available ducks. Every duck can quack and walk,
which are publically accessible functionalities of every duck. While every duck walks the same way, they quack differently which makes quacking an abstract feature of a duck. Every duck has a
weight (float) which determines their ability to float, which ability can be checked by everybody through canFloat method. The weight of a duck can be set through a method available only for all
concrete ducks implementations. A rubber duck, a kind of a duck, has all the features of a duck (yes, it quacks), but when asked to walk – it acts differently, because a rubber duck can't walk, so
it overrides a duck walk feature and does nothing. An example of a duck is also a mallard duck that apart of quacking and walking, can also fly. Implement classes, with names, attributes and methods along with proper relationship between classes. The main class Duck has weight in float,
have void walk(), quack(), Boolean canFloat(), void setWeight(float). The MallerDuck class can quack() and fly(). The RubberDuck class can quack() and fly().

# PROGRAM:

Project | oop lab 10 2.cpp

```cpp
#include <iostream>
using namespace std;

class Duck { //an abstract class//
protected:
    float weight;

public:
    virtual void quack() = 0; //pure virtual function//

    virtual void walk() {        //its just a normal function//
        cout << "Duck is walking" << endl;
    }

    // Check floating ability
    bool canFloat() {
        return weight < 5.0; // simple condition
    }

    void setWeight(float w) {  //setting the weight//
        weight = w;
    }
};

```

Project | oop lab 10 2.cpp

```cpp
class MallardDuck : public Duck {
public:
    void quack() {
        cout << "Mallard duck quacks" << endl;
    }

    void fly() {
        cout << "Mallard duck can fly" << endl;
    }
};

class RubberDuck : public Duck {
public:
    void quack() {
        cout << "Rubber duck squeaks" << endl;
    }

    void walk() {
        cout << "Rubber duck cannot walk" << endl;
    }

    void fly() {
        cout << "Rubber duck cannot fly" << endl;
    }
};
```

```cpp
};

int main() {
    MallardDuck m;
    m.setWeight(3.5);
    m.quack();
    m.walk();
    m.fly();
    cout << "Can float: " << m.canFloat() << endl;

    cout << endl;

    RubberDuck r;
    r.setWeight(1.0);
    r.quack();
    r.walk();
    r.fly();
    cout << "Can float: " << r.canFloat() << endl;

    return 0;
}
```

# OUTPUT:

C:\Users\MCS\OneDrive\Desktop\oop lab 10 2.exe

```
Mallard duck quacks
Duck is walking
Mallard duck can fly
Can float: 1

Rubber duck squeaks
Rubber duck cannot walk
Rubber duck cannot fly
Can float: 1

--------------------------------
Process exited after 0.02133 seconds with return value 0
Press any key to continue . . .
```

# TASK 3:

Many restaurants offer a soup-of-the-day. Good object-oriented customers will just order the soup-of-the-day and have faith that what is delivered will satisfy their hunger. Having a soup-of-the-day simplifies ordering for customers. They don't have to be experts on specific types of soups. The same request is made every time, but depending on which day the request is made, a
different type of soap is returned. Customers who order the soup-of-the-day are making a polymorphic request. The specific type of soup that arrives will depend on the menu of the day. To implement the above polymorphic scenario create a base class menu having virtual function soupOfTheDay() which returns “Sour Soup”. The menu class has a publically derived class weekDayMenu which override the menu class function and returns “Corn Soup”. Similarly, another publically derived class of menu override the virtual function and returns “Sweet soup”.
In main function create objects where required and call the desired functions to fulfill polymorphic
concept.

# PROGRAM:

Project [*] Untitled1

```
#include <iostream>
using namespace std;

// Base class
class Menu {
public:
    virtual string soupOfTheDay() {
        return "Sour Soup";
    }
};
// Derived class 1
class WeekDayMenu : public Menu {
public:
    string soupOfTheDay() {
        return "Corn Soup";
    }
};
// Derived class 2
class SpecialMenu : public Menu {
public:
    string soupOfTheDay() {
        return "Sweet Soup";
    }
};
int main() {
```

```
int main() {
    Menu* m;  // Base class pointer

    WeekDayMenu w;
    SpecialMenu s;

    m = &w;
    cout << "Weekday Soup: " << m->soupOfTheDay() << endl;

    m = &s;
    cout << "Special Soup: " << m->soupOfTheDay() << endl;

    return 0;
}
```

# OUTPUT:

```
C:\Users\MCS\OneDrive\Desktop\oop lab 10 3.exe
Weekday Soup: Corn Soup
Special Soup: Sweet Soup

--------------------------------
Process exited after 0.1326 seconds with return value 0
Press any key to continue . . .
```